

Principles of Computer Science II

From pseudocode to Python

Marco Zecchini

Sapienza University of Rome

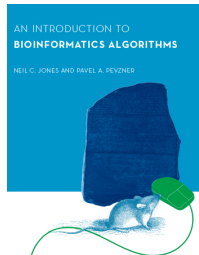
Tutorial 1

Today's plan

- A refresher of the programming concepts you met in PCSI, seen through the **pseudocode** of the textbook.
- For each concept: the pseudocode first, then the same thing in Python, then a short exercise.
- Why? In this course we *read* algorithms in pseudocode and *implement* them in Python.

Jones, Pevzner: An Introduction to Bioinformatics Algorithms. MIT Press, 2004.

Section 2.1, *What Is an Algorithm?*, pp. 8–11.



Before we start

Let us first see what Google Colab is and how to use it. You will do all the exercises in a Colab notebook. Then:

- 1 Open the Google Colab notebook of the tutorial (link on the course website); you need a Google account.
- 2 File > Save a copy in Drive, otherwise your changes are lost.
- 3 Run the cells **in order** with Shift+Enter. Something looks wrong? Runtime > Restart session and run all.
- 4 Every exercise has a cell to complete and a cell of **tests**: run the tests to check your solution.

Nothing to install: the notebook runs in the browser.

What is an algorithm?

An **algorithm** is a sequence of instructions that one must perform in order to solve a well-formulated problem.

- A problem is specified by its **input** and its **output**; the algorithm is the method that turns the input into the output.
- A human with pen and paper can carry out the steps, but humans are slow and make mistakes.
- A computer is fast and reliable, but cannot understand English: every detail must be spelled out in a **programming language**.

Pseudocode: neither too vague nor too formal

- **Pseudocode** lists the steps of an algorithm ignoring the details a programming language needs, yet it is more precise than a recipe in a cookbook.
- Ingredients: **variables** such as x or *total*, holding a value; **arrays** $\mathbf{a} = (a_1, a_2, \dots, a_n)$; **arguments**.
- An algorithm has a **name** followed by its arguments, e.g. $\text{MAX}(a, b)$; **return** reports the result or signals the end.
- Blocks are marked only by **indentation**: no “end if”, no “end for” ... exactly like Python.

Assignment: pseudocode

Format $a \leftarrow b$

Effect sets the variable a to the value b .

```
1 b ← 2
2 a ← b
```

Result: the value of a is 2 (and b is still 2).

Assignment: Python

```
b = 2
a = b
print(a)    # 2
```

- The arrow $\leftarrow$ becomes `=`: “store the value on the right in the variable on the left”.
- `=` is *not* equality: `a == b` is the question “are they equal?”.
- A variable is created the first time you assign to it; its type is not declared.
- **Try it:** what are `a` and `b` after `a = 5`? And after `a = a + 1`?

Arithmetic: pseudocode

Format $a + b$, $a - b$, $a \cdot b$, a/b , a^b

Effect addition, subtraction, multiplication, division and exponentiation of numbers.

DIST(x_1, y_1, x_2, y_2)

```
1 dx ←  $(x_2 - x_1)^2$ 
2 dy ←  $(y_2 - y_1)^2$ 
3 return  $\sqrt{dx + dy}$ 
```

Result: the Euclidean distance between the points (x_1, y_1) and (x_2, y_2) ; **DIST**(0, 0, 3, 4) returns 5.

Arithmetic: Python

```
def dist(x1, y1, x2, y2):  
    dx = (x2 - x1) ** 2  
    dy = (y2 - y1) ** 2  
    return (dx + dy) ** 0.5  
  
print(dist(0, 0, 3, 4))    # 5.0
```

- A named algorithm with arguments becomes a `def` with the same arguments; `return` is the same.
- Exponentiation is `**` (not `^`); a square root is `** 0.5` or `math.sqrt`.
- `/` always gives a decimal number: `7 / 2` is `3.5`. Integer division is `//` (`7 // 2` is `3`) and the remainder is `%` (`7 % 2` is `1`).

Exercise 1: change, with integer division

In Lecture 1 we met the **US Change** problem: given M cents, give the fewest coins among quarters (25), dimes (10), nickels (5) and pennies (1).

- Write a function `us_change(M)` that returns the number of quarters, dimes, nickels and pennies, using only assignments, `//` and `%`.
- Example: `us_change(77)` returns (3, 0, 0, 2).
- Hint: how many quarters fit in 77 cents? How many cents are left after that?

Conditional: pseudocode

Format:

```
if A is true
    B
else
    C
```

Effect: if statement *A* is true, executes instructions *B*, otherwise executes instructions *C*. The “else *C*” part may be omitted: then *B* is executed or not, depending on *A*.

MAX(*a*, *b*)

```
1 if a < b
2     return b
3 else
4     return a
```

Result: the maximum of *a* and *b*; MAX(1, 99) returns 99.

Conditional: Python

```
def maximum(a, b):  
    if a < b:  
        return b  
    else:  
        return a  
  
print(maximum(1, 99))    # 99
```

- if and else end with a colon; the block is the indented lines below (4 spaces).
- Conditions: <, <=, >, >=, ==, !=, combined with and, or, not.
- More than two cases: elif (“else if”).
- max already exists in Python, so we called ours maximum.

Exercise 2: the complementary base

In DNA, A pairs with T and C pairs with G.

- Write a function `complement(base)` that, given one nucleotide ('A', 'C', 'G' or 'T'), returns the complementary base.
- If the input is not a valid nucleotide, return the character '?'.
• Examples: `complement('A')` returns 'T';
`complement('X')` returns '?'.

for loops: pseudocode

Format:

```
for  $i \leftarrow a$  to  $b$   
     $B$ 
```

Effect: sets i to a and executes instructions B . Sets i to $a + 1$ and executes B again. Repeats for $i = a + 2, a + 3, \dots, b - 1, b$.

SUMINTEGERS(n)

```
1 sum  $\leftarrow$  0  
2 for  $i \leftarrow 1$  to  $n$   
3     sum  $\leftarrow$  sum +  $i$   
4 return sum
```

Result: the sum of the integers from 1 to n ; SUMINTEGERS(10) returns $1 + 2 + \dots + 10 = 55$.

for loops: Python

```
def sum_integers(n):  
    total = 0  
    for i in range(1, n + 1):  
        total = total + i  
    return total  
  
print(sum_integers(10))    # 55
```

- `range(a, b)` produces $a, a + 1, \dots, b - 1$: the last value is **excluded**, so “from 1 to n ” is `range(1, n + 1)`.
- `range(n)` starts from 0; `range(b, a - 1, -1)` counts down.
- `sum` is a Python built-in, so we used `total`.
- Python can also loop directly over the elements of a list or a string: `for base in "ACGT":`

Exercise 3: counting nucleotides

- Write a function `count_nucleotides(dna)` that, given a DNA string, returns how many times each of A, C, G and T appears (four numbers, in this order).
- Use one `for` loop over the characters of the string and four counters.
- Example: `count_nucleotides("ATGCGC")` returns (1, 2, 2, 1).
- Bonus: `gc_content(dna)`, the fraction of G and C over the length of the string (0.667 in the example).

while loops: pseudocode

Format:

```
while  $A$  is true  
     $B$ 
```

Effect: checks the condition A . If it is true, executes instructions B . Checks A again; if it is true, executes B again. Repeats until A is not true.

ADDUNTIL(b)

```
1  $i \leftarrow 1$   
2  $total \leftarrow i$   
3 while  $total \leq b$   
4      $i \leftarrow i + 1$   
5      $total \leftarrow total + i$   
6 return  $i$ 
```

Result: the smallest i such that $1 + \dots + i > b$; ADDUNTIL(25) returns 7 ($1 + \dots + 7 = 28 > 25$, $1 + \dots + 6 = 21$).

while loops: Python

```
def add_until(b):  
    i = 1  
    total = i  
    while total <= b:  
        i = i + 1  
        total = total + i  
    return i  
  
print(add_until(25))    # 7
```

- for when you know the number of iterations, while when you do not.
- The condition is checked **before** every iteration: if it is false at the start, the body never runs.
- The body must eventually make the condition false.

Exercise 4: bacterial growth

A bacterial culture starts with n_0 cells and the population doubles every 20 minutes.

- Write a function `time_to_reach(n0, target)` that returns after how many minutes the population first reaches at least `target` cells. Use a `while` loop, as in `ADDUNTIL`.
- Example: `time_to_reach(1, 1000)` returns 200 (after 10 doublings there are 1024 cells).
- Bonus: ask the user for `n0` and `target`, print the answer, and repeat until the user types `stop`.

Array access: pseudocode

Format a_i

Effect the i -th number of the array $\mathbf{a} = (a_1, \dots, a_i, \dots, a_n)$.
If $\mathbf{F} = (1, 1, 2, 3, 5, 8, 13)$, then $F_3 = 2$ and $F_4 = 3$.

FIBONACCI(n)

```
1  $F_1 \leftarrow 1$ 
2  $F_2 \leftarrow 1$ 
3 for  $i \leftarrow 3$  to  $n$ 
4      $F_i \leftarrow F_{i-1} + F_{i-2}$ 
5 return  $F_n$ 
```

Result: the n -th Fibonacci number; FIBONACCI(8) returns 21.

Array access: Python

```
def fibonacci(n):                # n >= 2
    F = [0] * (n + 1)           # n+1 slots, F[0] unused
    F[1] = 1
    F[2] = 1
    for i in range(3, n + 1):
        F[i] = F[i - 1] + F[i - 2]
    return F[n]

print(fibonacci(8))            # 21
```

- Arrays are **lists**: $F = [1, 1, 2, 3, 5, 8, 13]$; a_i becomes $F[i]$, but Python counts from **zero**: $F[0]$ is the first element.
- We allocate $n + 1$ slots to keep the book's indices; or start from $F = [1, 1]$ and append the sum of the last two.
- Reading past the end ($F[n + 1]$) is an error: `IndexError`.

Exercise 5: running total

- Write a function `running_total(a)` that, given a list of numbers `a`, returns a new list `b` of the same length where `b[i]` is the sum `a[0] + a[1] + ... + a[i]`.
- Build `b` element by element, using the previous element `b[i - 1]` as FIBONACCI does.
- Example: `running_total([3, 1, 4, 1, 5])` returns `[3, 4, 8, 9, 14]`.
- Bonus: without using `max()`, return the position (index) of the largest element of `a`.

Why not plain English?

- Biologists often describe procedures in the language of a cookbook: the book rewrites a pumpkin pie recipe (figure 2.1) as the pseudocode `MAKEPUMPKINPIE`.
- “The problem is that natural languages are not suitable for communicating algorithmic ideas more complex than the pumpkin pie.” (p. 11)
- “Computer scientists have yet to invent anything better than pseudocode for this purpose”, so the book uses it. And so will we.

From pseudocode to Python: cheat sheet

Pseudocode	Python
$a \leftarrow b$	<code>a = b</code>
$a \cdot b$, a^b , a/b	<code>a * b</code> , <code>a ** b</code> , <code>a / b</code> (also <code>//</code> , <code>%</code>)
if A ... else ...	<code>if A: ... else: ...</code> (and <code>elif</code>)
for $i \leftarrow a$ to b	<code>for i in range(a, b + 1):</code>
while A	<code>while A:</code>
a_i	<code>a[i - 1]</code> (lists start at 0)
<code>NAME(x, y) ... return</code>	<code>def name(x, y): ... return</code>

Blocks are marked by indentation in both; pseudocode has no colons, Python needs them.

Exercise 6: putting it all together

- Write **first the pseudocode**, then the Python function `longest_run(dna)` that returns the length of the longest stretch of identical consecutive nucleotides.
- Example: `longest_run("AATTTTGCCCA")` returns 4 (the stretch TTTT).
- Hint: scan the string with a `for` loop, keep a counter for the current stretch and one for the best so far; use an `if` to compare each nucleotide with the previous one.